

Document Number: P0057R00
Date: 2015-09-26
Revises: N4499
Authors: Gor Nishanov <gorn@microsoft.com>
Daveed Vandevoorde <daveed@edg.com>

Wording for Coroutines (Revision 3)

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

Contents

Contents	ii
List of Tables	iii
1 General	1
1.1 Scope	1
1.2 Acknowledgements	1
1.3 Normative references	1
1.4 Implementation compliance	1
1.5 Feature testing	1
1.9 Program execution	2
2 Lexical conventions	3
2.12 Keywords	3
3 Basic concepts	4
3.6 Start and termination	4
5 Expressions	5
5.3 Unary expressions	5
5.18 Assignment and compound assignment operators	6
5.21 Yield expressions	6
6 Statements	8
6.5 Iteration statements	8
6.6 Jump statements	9
7 Declarations	10
8 Declarators	12
8.4 Function definitions	12
12 Special member functions	16
12.8 Copying and moving class objects	16
13 Overloading	17
13.5 Overloaded operators	17
18 Language support library	18
18.1 General	18
18.10 Other runtime support	18
18.11 Coroutines support library	18

List of Tables

1	Feature-test macro	1
2	Language support library summary	18
3	Coroutine traits requirements	19
4	Descriptive variable definitions	22
5	<code>CoroutinePromise</code> requirements	23

1 General

[intro]

1.1 Scope

[intro.scope]

- ¹ This Technical Specification describes extensions to the C++ Programming Language (1.3) that enable definition of coroutines. These extensions include new syntactic forms and modifications to existing language semantics.
- ² The International Standard, ISO/IEC 14882, provides important context and specification for this Technical Specification. This document is written as a set of changes against that specification. Instructions to modify or add paragraphs are written as explicit instructions. Modifications made directly to existing text from the International Standard use underlining to represent added text and ~~strikethrough~~ to represent deleted text.
- ³ This revision updates N4499 with changes proposed in papers P0054 and P0070.

1.2 Acknowledgements

[intro.ack]

This work is the result of collaboration of researchers in industry and academia, including CppDes Microsoft group and the WG21 study group SG1. We wish to thank people who made valuable contributions within and outside these groups, including Jens Maurer, Artur Laksberg, Richard Smith, Chandler Carruth, Gabriel Dos Reis, Deon Brewis, Jonathan Caves, James McNellis, Stephan T. Lavavej, Herb Sutter, Pablo Halpern, Robert Schumacher, Michael Wong, Niklas Gustafsson, Nick Maliwacki, Vladimir Petter, Shahms King, Slava Kuznetsov, Tongari J, Lewis Baker, Lawrence Cowl, and many others not named here who contributed to the discussion.

1.3 Normative references

[intro.refs]

- ¹ The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
- (1.1) — ISO/IEC 14882:2014, *Programming Languages – C++*

ISO/IEC 14882:2014 is hereafter called the *C++ Standard*. Beginning with section 1.9 below, all clause and section numbers, titles, and symbolic references in [brackets] refer to the corresponding elements of the C++ Standard. Sections 1.1 through 1.5 of this Technical Specification are introductory material and are unrelated to the similarly-numbered sections of the C++ Standard.

1.4 Implementation compliance

[intro.compliance]

- ¹ Conformance requirements for this specification are the same as those defined in section 1.4 of the C++ Standard. [*Note: Conformance is defined in terms of the behavior of programs. — end note*]

1.5 Feature testing

[intro.features]

An implementation that provides support for this Technical Specification shall define the feature test macro in Table 1.

Table 1 — Feature-test macro

Name	Value	Header
<code>__cpp_coroutines</code>	201510	<i>predeclared</i>

1.9 Program execution

[intro.execution]

Modify paragraph 7 to read:

- ⁷ An instance of each object with automatic storage duration (3.7.3) is associated with each entry into its block. Such an object exists and retains its last-stored value during the execution of the block and while the block is suspended (by a call of a function, [suspension of a coroutine \(8.4.4\)](#), or receipt of a signal).

2 Lexical conventions [lex]

2.12 Keywords [lex.key]

[Editor's note: In Lenexa's EWG session there was a brief discussion on possible keywords. In this document we use placeholder keywords with suffix -keyword to be replaced with real ones in Kona. A companion paper discussing keyword alternatives is to appear in pre-Kona mailing.]

Add the keyword placeholders `await-keyword` and `yield-keyword` to Table 4 "Keywords".

3 Basic concepts

[basic]

3.6 Start and termination

[basic.start]

3.6.1 Main function

[basic.start.main]

Add underlined text to paragraph 3.

The function `main` shall not be used within a program. The linkage (3.5) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed. The function `main` shall not be a coroutine. The name `main` is not otherwise reserved. [*Example*: member functions, classes, and enumerations can be called `main`, as can entities in other namespaces. — *end example*]

5 Expressions

[expr]

5.3 Unary expressions

[expr.unary]

In this section change the grammar for *unary-expression* as follows:

```

unary-expression:
    postfix-expression
    ++ cast-expression
    -- cast-expression
    await-expression
    unary-operator cast-expression
    sizeof unary-expression
    sizeof ( type-id )
    sizeof ... ( identifier )
    alignof ( type-id )
    noexcept-expression
    new-expression
    delete-expression

```

5.3.7 noexcept operator

[expr.unary.noexcept]

In this section, add a new paragraph after paragraph 3.

- ⁴ If in a potentially-evaluated context the *expression* would contain a potentially-evaluated *await-expression*, the program is ill-formed.

5.3.8 Await

[expr.await]

Add this section to 5.3.

- ¹ The **await-keyword** operator is used to suspend evaluation of the enclosing coroutine (8.4.4) while awaiting completion of the computation represented by the operand expression.

```

await-expression:
    await-keyword cast-expression

```

- ² A potentially-evaluated *await-expression* shall only appear within the *compound-statement* of a *function-body* outside of a *handler* (15.3). In a *declaration-statement* or in the *simple-declaration* (if any) of a *for-init-statement*, a potentially-evaluated *await-expression* shall only appear in an *initializer* of that *declaration-statement* or *simple-declaration*. A potentially-evaluated *await-expression* shall not appear in a default argument (8.3.6).

- ³ Let *f* be an **operator await** function found according to the rules of operator function lookup for unary operators and **operator await** (13.5.9) for *cast-expression*. If such function cannot be found, the program is ill-formed. Let *e* be a temporary initialized as-if by **auto&& e = f(cast-expression)**; let *p* be the promise object (8.4.4) of the enclosing coroutine, *P* be the type of the promise object, *h* be an object of **std::experimental::coroutine_handle<P>** referring to the enclosing coroutine, and let *await-ready-expr*, *await-suspend-expr*, and *await-resume-expr* be *e.await_ready()*, *e.await_suspend(h)*, and *e.await_resume()*, respectively;

then

```

    await-keyword cast-expression

```

is equivalent to:


```
(
  await-ready-expr ? await-resume-expr
  : (await-suspend-expr, suspend-resume-point, await-resume-expr)
)
```

where *suspend-resume-points* are treated as expressions of type `void`. Suspend-resume-points are defined in (8.4.4).

4 An *await-expression* may only appear in a coroutine with an eventual return type (6.6.3.1). This constraint does not apply to implicit *await-expression* introduced as a result of evaluation of a *yield-expression* (5.21), initial, and final suspend points (8.4.4).

5 A coroutine is considered suspended just prior to the call to *await-suspend-expr*. If a return type of *await-suspend-expr* is `bool` and invocation of *await-suspend-expr* returns `false`, then execution continues as if coroutine was resumed. An exception thrown from within *await-suspend-expr* is propagated as if coroutine was resumed and an exception was thrown just prior to an invocation of *await-resume-expr*.

6 [Note: An *await-expression* may appear as an unevaluated operand (5.2.8, 5.3.3, 5.3.7, 7.1.6.2). The presence of such an *await-expression* does not make the enclosing function a coroutine and can be used to examine the type of an *await-expression*.

[Example:

```
std::future<int> f();

int main() {
  using t = decltype(await-keyword f()); // t is int
  static_assert(sizeof(await-keyword f()) == sizeof(int));
  cout << typeid(await-keyword f()).name() << endl;
}
```

— end example] — end note]

5.18 Assignment and compound assignment operators

[expr.ass]

In paragraph 1 add *yield-expression* to *assignment-expression* rule.

```
assignment-expression:
  conditional-expression
  logical-or-expression assignment-operator initializer-clause
  throw-expression
  yield-expression
```

5.21 Yield expressions

[expr.yield]

Add this section to Clause 5.

```
yield-expression:
  yield-keyword assignment-expression
  yield-keyword braced-init-list
```

1 Let *yielded value* be the operand of the **yield-keyword** expression and *p* be the promise object of the enclosing coroutine, then **yield** expression is evaluated as an **await** expression with operand *p.yield_value(yielded value)*. If coroutine promise does not define a function **yield_value**, the program is ill-formed. An implicit *await-expression* introduced by this evaluation is not considered an *await-expression* for the purpose of type deduction (7.1.6.4).

2 [Example:

```

using namespace std;
using namespace std::experimental;

generator<pair<int,int>> g1() {
    for (int i = 1; i < 10; ++i)
        yield-keyword {i,i};
}

auto g2() { // deduces to generator<pair<int,int>>
    for (int i = 1; i < 10; ++i)
        yield-keyword make_pair(i,i);
}

int main() {
    auto r1 = g1();
    auto r2 = g2();
    assert(equal(r1.begin(), r1.end(), r2.begin(), r2.end()));
}

```

— end example]

- 3 All restrictions on where an *await-expression* can appear apply to *yield-expressions*.

[Example:

```

    auto f(int x = yield 5); // ill-formed
    int a[] = { yield 1 }; // ill-formed

```

— end example]

6 Statements

[stmt.stmt]

6.5 Iteration statements

[stmt.iter]

Add underlined text to paragraph 1.

- ¹ Iteration statements specify looping.

iteration-statement:

```
while ( condition ) statement
do statement while ( expression ) ;
for ( for-init-statement conditionopt; expressionopt ) statement
for await-keywordopt ( for-range-declaration : for-range-initializer ) statement
```

6.5.4 The range-based for statement

[stmt.ranged]

Add underlined text to paragraph 1.

- ¹ For a range-based **for** statement of the form

```
for await-keywordopt ( for-range-declaration : expression ) statement
```

let *range-init* be equivalent to the *expression* surrounded by parentheses¹

```
( expression )
```

and for a range-based **for** statement of the form

```
for await-keywordopt ( for-range-declaration : braced-init-list ) statement
```

let *range-init* be equivalent to the *braced-init-list*. In each case, a range-based **for** statement is equivalent to

```
{
    auto && __range = range-init;
    for ( auto __begin = await-keywordopt begin-expr,
        __end = end-expr;
        __begin != __end;
        await-keywordopt ++__begin ) {
        for-range-declaration = *__begin;
        statement
    }
}
```

where await-keyword appears if and only if it appears immediately after the **for** keyword, and *__range*, *__begin*, and *__end* are variables defined for exposition only, and *_RangeT* is the type of the expression, and *begin-expr* and *end-expr* are determined as follows:

[Editor's note: The remainder of paragraph 1 remains unchanged and is not included here.]

¹) this ensures that a top-level comma operator cannot be reinterpreted as a delimiter between *init-declarators* in the declaration of *__range*.

6.6 Jump statements

[stmt.jump]

In paragraph 1 add four productions to the grammar:

```

jump-statement:
    break ;
    continue ;
    return expressionopt ;
    return braced-init-list ;
    return expressionopt ;
    return braced-init-list ;
    goto identifier ;

```

6.6.3 The return statement

[stmt.return]

Add underlined text to paragraph 1:

- 1 A function returns to its caller by the **return** statement. In this section, function refers to a function that is not a coroutine. The **return** statement in a coroutine described in section 6.6.3.1.

6.6.3.1 The return statement in a coroutine

[stmt.return.coroutine]

Add this section to 6.6.

- 1 A coroutine returns to its caller by the **return** statement or when suspended at a suspend-resume point (8.4.4).
- 2 If the promise type (8.4.4) of the coroutine defines the member function **return_void**, the coroutine is considered to have an *eventual return type* of **void**, if the promise type (8.4.4) of the coroutine defines the member function **return_value**, the coroutine is considered to have a non-**void** eventual return type, otherwise, the coroutine is considered not to have an eventual return type. If the promise type defines both **return_value** and **return_void** member functions, the program is ill-formed.
- 3 In this section, *p* refers to the promise object (8.4.4) of the enclosing coroutine.
- 4 A **return** statement with no operand shall be used only in coroutines without an eventual return type or with an eventual return type of **void**. In the latter case, completion of the coroutine is signaled to the promise of the coroutine by calling **p.return_void()**.
- 5 A **return** statement with an operand of type **void** shall be used only in functions without an eventual return type or with an eventual return type of **void**; in the former case, the operand is evaluated just before the call to **p.final_suspend()** (8.4.4); in the later case, the completion of the coroutine is signaled to the promise of the coroutine by calling **p.return_void()** and the operand is evaluated just prior to the call to **p.return_void()**.
- 6 A **return** statement with any other operand shall be used only in coroutines producing an eventual value; the completion of the coroutine is signalled to the promise by calling **p.return_value(*operand*)**. Flowing off the end of a coroutine is equivalent to a **return** with no value; this results in undefined behavior in a coroutine with non-**void** return type.

7 Declarations

[dcl.dcl]

7.1.5 The constexpr specifier

[dcl constexpr]

Add the underlined text as the last item in the list in paragraph 3. Note that the preceding (unmodified) items in the C++ Standard are elided in this document.

- 3 The definition of a `constexpr` function shall satisfy the following constraints:
- (3.1) — ...
 - (3.2) — ...
 - (3.3) — ...
 - (3.4) — ...
 - (3.5) — it shall not be a coroutine (8.4.4);

7.1.6.4 auto specifier

[dcl.spec.auto]

Add the underlined text to paragraph 2.

- 2 The placeholder type can appear with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such a declarator is valid. If the function declarator includes a *trailing-return-type* (8.3.5), that specifies the declared return type of the function. If the declared return type of the function contains a placeholder type, the return type of the function is deduced from `return` statements and *yield-expressions* in the body of the function, if any.

Add the underlined text to paragraph 9.

- 9 If a function with a declared return type that contains a placeholder type has multiple `return` statements, and *yield-expressions*, the return type is deduced for each `return` statement and *yield-expression*. If the type deduced is not the same in each deduction, the program is ill-formed.

Add paragraphs 16 through 18.

- 16 If a coroutine has a declared return type that contains a placeholder type, then the return type of the coroutine is deduced as follows:
- (16.1) — If both a *yield-expression* and an *await-expression* are present, then the return type is `std::experimental::async_stream<T>`, where T is deduced from the *yield-expressions* as if a *yield-expression* were a `return` statement in a function with declared type `auto` without a *trailing-return-type*.
 - (16.2) — Otherwise, if an *await-expression* is present in a function, then the return type is `std::experimental::task<T>` where type T is deduced from `return` statements as if the `return` statements were in a function with declared type `auto` without a *trailing-return-type*.
 - (16.3) — Otherwise, if a *yield-expression* is present in a function, then the return type is `std::experimental::generator<T>`, where T is deduced from the *yield-expressions* as if a *yield-expression* were a `return` statement in a function with declared type `auto` without a *trailing-return-type*.

[Example:

```

// deduces to std::experimental::generator<char>
auto f() { for(auto ch: "Hello") yield-keyword ch; }

// deduces to std::experimental::async_stream<int>
auto ticks() {
    for(int tick = 0;; ++tick) {
        yield-keyword tick;
        await-keyword sleep_for(1ms);
    }
}

future<void> g();

// deduces to std::experimental::task<void>
auto f2() { await-keyword g(); }

// deduces to std::experimental::task<int>
auto f3() {
    await-keyword g();
    return 42;
}

```

— *end example*]

17

The templates `std::experimental::generator`, `std::experimental::task`, and `std::experimental::async_stream` are not predefined; if the appropriate headers are not included prior to a use — even an implicit use in which the type is not named (7.1.6.4) — the program is ill-formed.

8 Declarators

[dcl.decl]

8.3.5 Functions

[dcl.fct]

Add paragraph 16.

- ¹⁶ If the *parameter-declaration-clause* terminates with an ellipsis that is not part of *abstract-declarator*, a function shall not be coroutine (8.4.4).

8.4 Function definitions

[dcl.fct.def]

8.4.4 Coroutines

[dcl.fct.def.coroutine]

Add this section to 8.4.

- ¹ A function is a *coroutine* if it contains one or more suspend-resume-points introduced by a potentially-evaluated *await-expression* (5.3.8). Every coroutine also has an implicit initial and final suspend-resume point as described later in this section.

- ² [Note: From the perspective of the caller, a coroutine is just a function with that particular signature. The fact that a function is implemented as a coroutine is unobservable by the caller. — end note]

- ³ [Example:

```
// coroutine hello world
std::experimental::generator<char> hello_fn() {
    for (auto ch: "Hello, world") yield-keyword ch;
}

int main() {
    // coroutine as a lambda
    auto hello_lambda = []{ for (auto ch: "Hello, world") yield-keyword ch; };

    for (auto ch : hello_lambda()) cout << ch;
    for (auto ch : hello_fn()) cout << ch;
}
```

— end example]

- ⁴ A coroutine needs a set of related types and functions to complete the definition of its semantics. These types and functions are provided as a set of member types or typedefs and member functions in the specializations of class template `std::experimental::coroutine_traits` (18.11.1).

- ⁵ For a coroutine f , if f is a non-static member function, let P_1 denote the type of the implicit object parameter (13.3.1) and $P_2 \dots P_n$ be the types of the function parameters; otherwise let $P_1 \dots P_n$ be the types of the function parameters. Let R be the return type and F be the *function-body* of f , T be a type `std::experimental::coroutine_traits< $R, P_1, \dots, P_n$ >`, and P be the type denoted by `T::promise_type`. Then, the coroutine behaves as if its body were:

```
{
    P p;
    await-keyword p.initial_suspend(); // initial suspend point
    F'
    await-keyword p.final_suspend(); // final suspend point
}
```

where local variable *p* is defined for exposition only and *F'* is *F* if *P* does not define a `set_exception` member function, and

```
try { F } catch(...) { p.set_exception(std::current_exception()); }
```

otherwise. No header needs to be included for this use of the function `std::current_exception`. An object denoted as *p* is the *promise object* of the coroutine *f* and its type is a *promise type* of the coroutine. An execution of a coroutine is suspended when it reaches a suspend-resume-point. Implicit *await-expressions* introducing initial and final suspend points are not considered *await-expressions* for the purpose of type deduction (7.1.6.4).

6 A *suspension* of a coroutine returns control to the current caller of the coroutine. For the first return of control from the coroutine, the return value is obtained by invoking the member function `get_return_object` (18.11.4) of the promise object.

7 A suspended coroutine can be resumed to continue execution by invoking a resumption member functions (18.11.2.4) of an object of `coroutine_handle<P>` type associated with this instance of the coroutine, where type *P* is the promise type of the coroutine. Invoking resumption member functions in a coroutine that is not suspended results in undefined behavior.

8 A coroutine may need to allocate memory to store objects with automatic storage duration local to the coroutine. If so, it shall obtain the storage by calling an *allocation function* (3.7.4.1). The allocation function's name is looked up in the scope of the promise type of the coroutine. If this lookup fails to find the name, the allocation function's name is looked up in the global scope. If the lookup finds an allocation function that takes exactly one parameter, it will be used, otherwise, all parameters passed to the coroutine are passed to the allocation function after the size parameter. [Note: The second form allows coroutines to support stateful allocators. — end note]

9 [Example:

```
class Arena;
struct my_coroutine {
    struct promise_type {
        ...
        template <typename... TheRest>
        void* operator new(std::size_t size, Arena* pool, TheRest const&...) {
            return pool.allocate(size);
        }
    };
};

my_coroutine (Arena* a) {
    // will call my_coroutine::promise_type::new(<required-size>, a)
    // to obtain storage for the coroutine state
    yield 1;
}

int main() {
    Pool memPool;
    for (int i = 0; i < 1'000'000; ++i)
        my_coroutine(memPool);
};
```

— end example]

10 A *coroutine state* consists of storage for objects with automatic storage duration that are live at the current point of execution or suspension of a coroutine. The coroutine state is destroyed

when the control flows off the end of the function or the `destroy` member function (18.11.2.4) of an object of `std::experimental::coroutine_handle<P>` associated with that coroutine is invoked. In the latter case objects with automatic storage duration that are in scope at the suspend point are destroyed in the reverse order of the construction. If the coroutine state required dynamic allocation, the storage is released by calling a deallocation function (3.7.4.2). If `destroy` is called for a coroutine that is not suspended, the program has undefined behavior.

- 11 The deallocation function's name is looked up in the scope of the coroutine promise type. If this lookup fails to find the name, the deallocation function's name is looked up in the global scope. If deallocation function lookup finds both a usual deallocation function with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter, then the selected deallocation function shall be the one with two parameters. Otherwise, the selected deallocation function shall be the function with one parameter.
- 12 When a coroutine is invoked, each of its parameters is copied/moved to the coroutine state, as specified in 12.8. The copy/move operations are indeterminately sequenced with respect to each other. A reference to a parameter in the function-body of the coroutine is replaced by a reference to the copy of the parameter.
- 13 If during the coroutine state initialization, a call to `get_return_object`, or a promise object construction throws an exception, objects with automatic storage duration (3.7.3) that have been constructed are destroyed in the reverse order of their construction, any memory dynamically allocated for the coroutine state is freed and the search for a handler starts in the scope of the calling function.
- 14 If type *P* defines static member function `get_return_object_on_allocation_failure` (18.11.1), then `std::nothrow_t` forms of allocation and deallocation functions will be used. If an allocation function returns `nullptr`, coroutine must return control to the caller of the coroutine and the return value shall be obtained by a call to `P::get_return_object_on_allocation_failure()`.

[*Example:*

```
struct generator {
    struct promise_type {
        int current_value;
        static auto get_return_object_on_allocation_failure() { return generator{nullptr}; }
        auto get_return_object() { return generator{this}; }
        auto initial_suspend() { return std::experimental::suspend_always{}; }
        auto final_suspend() { return std::experimental::suspend_always{}; }
        auto yield_value(int value) {
            current_value = value;
            return std::experimental::suspend_always{};
        }
    };
    bool move_next() { return coro ? (coro.resume(), !coro.done()) : false; }
    int current_value() { return coro.promise().current_value; }
    ~generator() { if(coro) coro.destroy(); }
private:
    generator(promise_type* myPromise) : coro(myPromise) {}
    std::experimental::coroutine_handle<promise_type> coro;
};

generator f() { yield-keyword 1; yield-keyword 2; }

int main() {
    auto g = f();
    while (g.move_next()) std::cout << g.current_value() << std::endl;
}
```

— *end example*]

12 Special member functions [special]

In this section add new paragraph after paragraph 5.

- 6 A special member function shall not be a coroutine.

12.8 Copying and moving class objects [class.copy]

Add underlined text to paragraph 31.

- 31 When certain criteria are met, an implementation is allowed to omit the copy/move construction of a class object, even if the constructor selected for the copy/move operation and/or the destructor for the object have side effects. In such cases, the implementation treats the source and target of the omitted copy/move operation as simply two different ways of referring to the same object, and the destruction of that object occurs at the later of the times when the two objects would have been destroyed without the optimization.² This elision of copy/move operations, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):
- (31.1) — in a **return** statement in a function with a class return type, when the expression is the name of a non-volatile automatic object (other than a function or catch-clause parameter) with the same cv-unqualified type as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function's return value
 - (31.2) — When a parameter would be copied/moved to the coroutine state (8.4.4) copy move can be omitted by continuing to refer to the function parameters instead of referring to their copies in the coroutine state.

²) Because only one object is destroyed instead of two, and one copy/move constructor is not executed, there is still one object destroyed for each one constructed.

13 Overloading

[over]

13.5 Overloaded operators

[over.oper]

Add `await`-keyword to the list of operators in paragraph 1 before operators `()` and `[]`.

13.5.9 Await operator

[over.await]

- ¹ If there is no user-declared `operator await` for type `X`, but there is a declared member with a name `await_suspend`, `await_ready`, or `await_resume`, then the implementation shall provide the implicit definition of `operator await` in such a way that the result of the evaluation of an implicit `operator await(v)` is `v` itself.

18 Language support library

[language.support]

18.1 General

[support.general]

Add a row to Table 2 for coroutine support header `<experimental/coroutine>`.

Table 2 — Language support library summary

Subclause		Header(s)
18.2	Types	<code><cstdint></code>
18.3	Implementation properties	<code><limits></code>
		<code><climits></code>
		<code><cfloat></code>
18.4	Integer types	<code><cstdint></code>
18.5	Start and termination	<code><cstdlib></code>
18.6	Dynamic memory management	<code><new></code>
18.7	Type identification	<code><typeinfo></code>
18.8	Exception handling	<code><exception></code>
18.9	Initializer lists	<code><initializer_list></code>
18.11	Coroutines support	<experimental/coroutine>
18.10	Other runtime support	<code><csignal></code>
		<code><setjmp></code>
		<code><stdalign></code>
		<code><stdarg></code>
		<code><stdbool></code>
		<code><stdlib></code>
		<code><ctime></code>

18.10 Other runtime support

[support.runtime]

Add underlined text to paragraph 4.

- ⁴ The function signature `longjmp(jmp_buf jbuf, int val)` has more restricted behavior in this International Standard. A `setjmp/longjmp` call pair has undefined behavior if replacing the `setjmp` and `longjmp` by `catch` and `throw` would invoke any non-trivial destructors for any automatic objects. [A call to `setjmp` or `longjmp` has undefined behavior if invoked in a coroutine.](#)
- SEE ALSO: ISO C 7.10.4, 7.8, 7.6, 7.12.

18.11 Coroutines support library

[support.coroutine]

Add this section to clause 18.

- ¹ The header `<experimental/coroutine>` defines several types providing compile and run-time support for coroutines in a C++ program.

Header `<experimental/coroutine>` synopsis

```
namespace std {
    namespace experimental {
```

```

inline namespace coroutines_v1 {
    // 18.11.1 coroutine traits
    template <typename R, typename... ArgTypes>
        class coroutine_traits;

    // 18.11.2 coroutine handle
    template <typename Promise = void>
        class coroutine_handle;

    // 18.11.2.7 comparison operators:
    bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;

    // 18.11.3 trivial awaitables
    struct suspend_never;
    struct suspend_always;

} // namespace coroutines_v1
} // namespace experimental

// 18.11.2.8 hash support:
template <class T> struct hash;
template <class P> struct hash<experimental::coroutine_handle<P>>;
} // namespace std

```

18.11.1 coroutine traits [coroutine.traits]

1 This subclause defines requirements on classes representing *coroutine traits*, and defines the class template `coroutine_traits` that satisfies those requirements.

2 The `coroutine_traits` may be specialized by the user to customize the semantics of coroutines.

18.11.1.1 Coroutine traits requirements [coroutine.traits.requirements]

1 In Table 3, `X` denotes a trait class instantiated as described in 8.4.4.

Table 3 — Coroutine traits requirements [tab:coroutine.traits.requirements]

Expression	Behavior
<code>X::promise_type</code>	<code>X::promise_type</code> must be a type satisfying coroutine promise requirements (18.11.4)

18.11.1.2 Struct template `coroutine_traits` [coroutine.traits.primary]

1 The header `<experimental/coroutine>` shall define the class template `coroutine_traits` as follows:

```

namespace std {
    namespace experimental {
        inline namespace coroutines_v1 {
            template <typename R, typename... Args>
                struct coroutine_traits {

```

```

    using promise_type = typename R::promise_type;
};
} // namespace coroutines_v1
} // namespace experimental
} // namespace std

```

18.11.2 Struct template `coroutine_handle`

[`coroutine.handle`]

```

namespace std {
    namespace experimental {
        inline namespace coroutines_v1 {
            template <>
            struct coroutine_handle<void>
            {
                // 18.11.2.1 construct/reset
                constexpr coroutine_handle() noexcept;
                constexpr coroutine_handle(nullptr_t) noexcept;
                coroutine_handle& operator=(nullptr_t) noexcept;

                // 18.11.2.2 export/import
                static coroutine_handle from_address(void* addr) noexcept;
                void* to_address() const noexcept;

                // 18.11.2.3 capacity
                explicit operator bool() const noexcept;

                // 18.11.2.4 resumption
                void operator()() const;
                void resume() const;
                void destroy() const;

                // 18.11.2.5 completion check
                bool done() const noexcept;
            };

            template <typename Promise>
            struct coroutine_handle : coroutine_handle<>
            {
                // 18.11.2.1 construct/reset
                using coroutine_handle<>::coroutine_handle;
                coroutine_handle(Promise*) noexcept;
                coroutine_handle& operator=(nullptr_t) noexcept;

                // 18.11.2.6 promise access
                Promise& promise() noexcept;
                Promise const& promise() const noexcept;
            };
        } // namespace coroutines_v1
    } // namespace experimental
} // namespace std

```

- ¹ Let P be a promise type of the coroutine (8.4.4). An object of the type `coroutine_handle<P>` is called a *coroutine handle* and can be used to refer to a suspended or executing coroutine. Such a function is called a *target* of a coroutine handle. A default constructed `coroutine_handle`

object has no target.

18.11.2.1 coroutine_handle construct/reset [coroutine.handle.con]

```
constexpr coroutine_handle() noexcept;
constexpr coroutine_handle(nullptr_t) noexcept;
```

1 *Postconditions:* `!*this`.

```
coroutine_handle(Promise* p) noexcept;
```

2 *Requires:* `p` points to a promise object of a coroutine.

3 *Postconditions:* `!*this` and `addressof(this->promise()) == p`.

```
coroutine_handle& operator=(nullptr_t) noexcept;
```

4 *Postconditions:* `!*this`.

5 *Returns:* `*this`.

18.11.2.2 coroutine_handle export/import [coroutine.handle.export]

```
static coroutine_handle from_address(void* addr) noexcept;
void* to_address() const noexcept;
```

1 *Postconditions:* `coroutine_handle<>::from_address(this->to_address()) == *this`.

18.11.2.3 coroutine_handle capacity [coroutine.handle.capacity]

```
explicit operator bool() const noexcept;
```

1 *Returns:* `true` if `*this` has a target, otherwise `false`.

18.11.2.4 coroutine_handle resumption [coroutine.handle.resumption]

```
void operator()() const;
void resume() const;
```

1 *Requires:* `*this` refers to a suspended coroutine.

2 *Effects:* resumes the execution of a target function. If the function was suspended at the final suspend point, `terminate` is called (15.5.1).

```
void destroy() const;
```

3 *Requires:* `*this` refers to a suspended coroutine.

4 *Effects:* destroys the target coroutine (8.4.4).

18.11.2.5 coroutine_handle completion check [coroutine.handle.completion]

```
bool done() const noexcept;
```

1 *Requires:* `*this` refers to a suspended coroutine.

2 *Returns:* `true` if the target function is suspended at final suspend point, otherwise `false`.

18.11.2.6 coroutine_handle promise access [coroutine.handle.prom]

```
Promise& promise() noexcept;
Promise const& promise() const noexcept;
```

1 *Requires:* `*this` refers to a coroutine.

2 *Returns:* a reference to a promise of the target function.

18.11.2.7 Comparison operators**[coroutine.handle.compare]**

```
bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
1     Returns: x.to_address() == y.to_address().
```

```
bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
2     Returns: x.to_address() < y.to_address().
```

```
bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
3     Returns: !(x == y).
```

```
bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
4     Returns: (y < x).
```

```
bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
5     Returns: !(x > y).
```

```
bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

```
6     Returns: !(x < y).
```

18.11.2.8 Hash support**[coroutine.handle.hash]**

```
template <class P> struct hash<experimental::coroutine_handle<P>>;
```

```
1     The template specializations shall meet the requirements of class template hash (20.9.12).
```

18.11.3 trivial awaitables**[coroutine.trivial.awaitables]**

The header `<experimental/coroutine>` shall define `suspend_never` and `suspend_always` as follows.

```
struct suspend_never {
    bool await_ready() { return false; }
    void await_suspend(coroutine_handle<>){}
    void await_resume() {}
};
struct suspend_always {
    bool await_ready() { return true; }
    void await_suspend(coroutine_handle<>){}
    void await_resume() {}
};
```

18.11.4 Coroutine promise requirements**[coroutine.promise]**

```
1     A user supplies the definition of the coroutine promise to implement desired high-level semantics
    associated with a coroutines discovered via instantiation of struct template coroutine_traits.
    The following tables describe the requirements on coroutine promise types.
```

Table 4 — Descriptive variable definitions

Variable	Definition
P	a coroutine promise type
p	a value of type P
e	a value of <code>exception_ptr</code> type
h	a value of <code>experimental::coroutine_handle<P></code> type
v	an <i>expression</i> or <i>braced-init-list</i>

Table 5 — CoroutinePromise requirements [CoroutinePromise]

Expression	Note
P{}	Construct an object of type P
p.get_return_object()	The <code>get_return_object</code> is invoked by the coroutine to construct the return object prior to reaching the first suspend-resume point, a <code>return</code> statement, or flowing off the end of the function.
P::get_return_object_on_allocation_failure()	(Optional) The <code>get_return_object_on_allocation_failure</code> is invoked by the coroutine to construct the return object if allocation of the coroutine state failed.
p.return_value(v)	(Optional) Invoked by a coroutine when a <code>return</code> statement with an operand is encountered in a coroutine as described in (6.6.3.1).
p.return_void()	(Optional) Invoked when a <code>return</code> statement is encountered as described in (6.6.3.1).
p.set_exception(e)	(Optional) If present, the <code>set_exception</code> is invoked by a coroutine when an unhandled exception occurs within a <i>function-body</i> of the coroutine. If the promise does not provide <code>set_exception</code> , an unhandled exception will propagate from the coroutine normally.
p.yield_value(v)	(Optional) The <code>yield_value</code> is invoked when <code>yield-keyword</code> statement is encountered in the coroutine. If promise does not define <code>yield_value</code> , <code>yield-keyword</code> statement shall not appear in the coroutine body.
p.initial_suspend()	A coroutine invokes <code>p.initial_suspend()</code> to obtain awaitable for <i>initial suspend point</i> (8.4.4).
p.final_suspend()	A coroutine invokes <code>p.final_suspend()</code> to obtain awaitable for <i>final suspend point</i> (8.4.4).

² [Example: This example illustrates full implementation of a promise type for a simple generator.

```
#include <iostream>
#include <experimental/coroutine>

struct generator {
    struct promise_type {
        int current_value;
        auto get_return_object() { return generator{this}; }
        auto initial_suspend() { return std::experimental::suspend_always{}; }
        auto final_suspend() { return std::experimental::suspend_always{}; }
        auto yield_value(int value) {
            current_value = value;
            return std::experimental::suspend_always{};
        }
    };

    bool move_next() {
        coro.resume();
        return !coro.done();
    }

    int current_value() { return coro.promise().current_value; }
};
```

```
    ~generator() { coro.destroy(); }
private:
    explicit generator(promise_type* myPromise) : coro(myPromise)
    {
    }
    std::experimental::coroutine_handle<promise_type> coro;
};

generator f() {
    yield-keyword 1;
    yield-keyword 2;
}

int main() {
    auto g = f();
    while (g.move_next()) std::cout << g.current_value() << std::endl;
}

— end example]
```